

=====

Problem 1

The objective is to find the root(s) for $f(x) = x^6 - x - 1$ on the interval $[a, b] = [1, 2]$. That is, finding x such that $f(x) = 0$. This will be done using the bisection method.

- a) One of the important assumptions behind root-finding in general is that the function changes signs on the given interval. That is, a root actually exists. This can be shown for the problem at hand by the following:

$$f(a) = f(1) = 1^6 - 1 - 1 = -1,$$

$$f(b) = f(2) = 2^6 - 2 - 1 = 61, \text{ meaning}$$

$$f(a)f(b) < 0.$$

Therefore, the bisection method can be applied because the function changes sign on the interval. That is, there is a point where it passes through the x axis, a root.

- b) The first four iterations of the bisection methods were computed below:

Bisection method for $x^6 - x - 1$ on $[1, 2]$, first four iterations

n	a_n	b_n	c_n	$f(a_n)$	$f(c_n)$	$f(a_n)f(c_n)$	$[a_{n+1}, b_{n+1}]$
0	1	2	1.5	-1	8.890625	-8.890625	$[1, 1.5]$
1	1	1.5	1.25	-1	1.564647	-1.564647	$[1, 1.25]$
2	1	1.25	1.125	-1	-0.097713	0.097713	$[1.125, 1.25]$
3	1.125	1.25	1.1875	-0.097713	0.616653	-0.060255	$[1.125, 1.1875]$
4	1.125	1.1875	1.15625	-0.097713	0.233268	-0.022793	$[1.125, 1.15625]$

↑ root estimates

Intuition would suggest that the best approximation is given by the final iteration, where $x = 1.15625$. One might support this as the best estimate because the interval width is the smallest at this final iteration. That is, $|b_n - a_n|$ is closest to 0 at $n = 4$. It should be noted that the functional values $f(a_n)$ and $f(c_n)$ were truncated to six decimal places to fit in the table. This truncation has no effect on the outcome because these are only used to determine the interval at the next step $[a_{n+1}, b_{n+1}]$ by observing the sign of their product $f(a_n) \cdot f(c_n)$ (and checking whether it is equal to 0).

=====

- c) The true solution of the root-finding problem is known to be $\alpha = 1.1347241384015195$. Using this value, errors for each iteration were calculated via subtraction. For analysis purposes, the ratio between errors was also calculated to observe their behavior.

Iteration	Estimate c_n	Error $E_n = \alpha - c_n $	Ratio $R_n = \frac{E_{n-1}}{E_n}$
0	1.5	0.365275861598	-
1	1.25	0.115275861598	3.168711
2	1.125	0.009724138401	11.85461
3	1.1875	0.052775861598	0.184253
4	1.15625	0.021525861598	2.45174

The errors appear to decrease at certain steps but increase at others. The smallest error is at $n = 2$, indicating the estimate is closest at this iteration. This is further supported by the ratio at this iteration, which shows the estimate rapidly approached the true value. However, the error increases at $n = 3$ before decreasing again at the following iteration as a result of the sign change of $f(a_2) \cdot f(c_2)$. This led to a different interval, namely a different lower bound. Despite the error increasing, it would be expected for the error to approach 0 as the interval $[a_n, b_n]$ is smaller at each step (as mentioned in the previous part). Since the interval is being cut in half at each iteration, and $b_n - a_n$ is a constant, we expect the interval width to approach 0 as $n \rightarrow \infty$. More formally:

$$\text{Given } c_n = \frac{a_n + b_n}{2},$$

$$|b_n - a_n| = \frac{1}{2}|b_{n-1} - a_{n-1}| = \frac{1}{2} \frac{1}{2} |b_{n-2} - a_{n-2}| = \left(\frac{1}{2}\right)^n (b - a)$$

$$\lim_{n \rightarrow \infty} \left(\frac{1}{2}\right)^n (b - a) = 0$$

“=====”

=====

Problem 2

The objective here is the same as the first problem, but this will use a different approach.

Namely, the Newton method with initial guess $x_0 = 2$.

The image below shows the hand-calculations for x_n , with results formatted into the following table. Additionally, the table includes values for the error of each iterate along with two ratios which will be used for analysis. Error calculations used true solution $\alpha = 1.1347241384015195$.

Newton method for $f(x) = x^6 - x - 1$, initial guess $x_0 = 2$, first 2 iterations

$f'(x) = 6x^5 - 1$, $x_{n+1} = x_n - f(x_n)/f'(x_n)$

n	x_n	$f(x_n)$	$f'(x_n)$	x_{n+1}
0	2	61	191	1.680628272
1	1.680628272	19.85294112	79.44694863	1.430738898
2	1.430738898	6.14679547	34.97106615	1.254970956

Iteration	Estimate x_n	Error $ \alpha - x_n $	$R_1 = \frac{\alpha - x_n}{\alpha - x_{n-1}}$	$R_2 = \frac{\alpha - x_n}{(\alpha - x_{n-1})^2}$
0	2	0.86527586159	-	-
1	1.680628272	0.54590413359	0.63090184046	0.72913375775
2	1.430738898	0.29601475959	0.54224678176	0.99330037713

The errors of the estimates appear to be approaching 0, as well as the ratio between errors. The ratio between the error of the current and the previous estimate error squared appears to be approaching 1, though this may be difficult to discern with only one step in between.

=====

“=====”

Problem 3

Now, the objective is to approach the root-finding problem with coding using Octave. This problem attempts to find the root for the same $f(x)$ as the previous problems, with known solution $\alpha = 1.1347241384015195$.

The code was written in two separate files, one consisting solely of the function for Newton's Method, and the other simply to call the function. The following code was used to set up the variables and function handles used in the separate file, along with a string to neatly organize the table. "Format long" was used to give a double-precision result for the solution once the algorithm was sufficiently close to the true value. The initial guess for the root was 2 as in the previous problem, and the tolerance was chosen to be 0.001 with maximum iterations being 100. The tolerance can readily be decreased by including more zeroes in its definition, giving a more precise result that is accurate to a larger number of decimal places.

Additionally, R1 will be defined as $\frac{\alpha - x_n}{\alpha - x_{n-1}}$, and R2 as $\frac{\alpha - x_n}{(\alpha - x_{n-1})^2}$. These will be assembled in a

table along with each root estimate, its corresponding functional value, and errors between estimates. This table will be used to analyze the behavior of the algorithm.

```
1| # Alex Caramanico April 19, 2023
2| format long
3| fhandle = @(x) (x^6-x-1);
4| jhandle = @(x) (6*x^5-1);
5| x0 = 2; tol = 0.001; maxit=100; true_x = 1.1347241384015195;
6| disp(' n      xn      f(xn)      True Err      Approx Err      R1      R2')
7| newton(fhandle,jhandle,x0,tol,maxit,true_x)
```

The function in the newton.m file that follows takes in parameters:

- *fhandle* - function of x for evaluating $f(x)$ at each root estimate within octave
- *jhandle* - function of x for evaluating $f'(x)$ at each root estimate within octave
- *x0* - initial guess for the root of the function
- *tol* - the minimum decrease between successive root estimates
- *maxit* - the maximum number of iterations for the algorithm to prevent infinite loops
- α - true solution, unavailable in many problems but used for analysis in this case

The first lines of newton.m set up the base conditions. The values are initialized and the first row of the table is calculated and printed. A variable for formatting the following rows is initialized, followed by a while loop containing the primary structure of the algorithm. The loop is conditioned to converge the successive root estimates until the tolerance is reached or the maximum iterations are reached (set by variable maxit). Inside of the while loop the functional values for $f(x)$ and $f'(x)$ are calculated for each value of x .

Following a formatting variable for successive rows of the table, a while loop is used as the primary structure of the algorithm. The loop continues until the algorithm converges (that is, successive root estimates are sufficiently close together), or the maximum number of iterations is reached. The first step of the loop is to increase the number of iterations by 1 at each loop.

Next, functional values for $f(x)$ and $f'(x)$ are calculated for each value of x . Then, Newton's algorithm is applied to generate a new root estimate. Using the new estimate, errors are calculated using both the true solution α and previous estimate prev_x . The ratios are then calculated and all pertinent information is formatted into rows of the table. After this point the code returns to the top of the while loop and checks the conditions again, repeating the algorithm if necessary. The conditional at the end is in place to warn the user that the algorithm did not converge within the number of iterations specified.

```

1| function [x, iter] = newton(fhhandle,jhandle,x0,tol,maxit,true_x);
2| x = x0; # initialize values
3| err = tol + 1;
4| iter = 0;
5| true_err = abs(true_x-x); # initial true error
6| row1form = '%3i %10.10e %10.10e %10.10e \n';
7| fprintf(row1form, iter, x, fhhandle(x), true_err)
8| rowsform = '%3i %10.10e %10.10e %10.10e %10.10e %10.10e %10.10e \n';
9| while (err > tol) && (iter < maxit);
10| iter +=1;
11| prev_x = x; # keep previous root estimate
12| f = fhhandle(x); # f(xn)
13| F = jhandle (x); # f'(xn)
14| x = prev_x - f/F; # Newton algorithm
15| fnnew = fhhandle(x) # functional value at new x
16| err = abs(x - prev_x); # error between estimates
17| true_err = abs(true_x - x); # error from true value
18| R1 = true_err / abs(true_x - prev_x); # Ratio between true errors
19| R2 = true_err / (true_x - prev_x)^2;

20| fprintf(rowsform, iter, x, fnnew, true_err, err, R1, R2)
21| endwhile
22| if (iter == maxit);
23| warning('Maximum Iterations Reached')
24| endif
25| endfunction

```

The code produced the following table and final estimate (ans) for the root. Analysis of the results follows on the next page.

n	xn	f (xn)	True Err	Approx Err	R1	R2
0	2.0000000000e+00	6.1000000000e+01	8.6527586160e-01			
1	1.6806282723e+00	1.9852941124e+01	5.4590413385e-01	3.1937172775e-01	6.3090184076e-01	7.2913375810e-01
2	1.4307389882e+00	6.1467954843e+00	2.9601484984e-01	2.4988928401e-01	5.4224694682e-01	9.9330067900e-01
3	1.2549709561e+00	1.6516569690e+00	1.2024681771e-01	1.7576803213e-01	4.0621886967e-01	1.3722922005e+00
4	1.1615384328e+00	2.9430969112e-01	2.6814294372e-02	9.3432523336e-02	2.2299379628e-01	1.8544673409e+00
5	1.1363532742e+00	1.6826073996e-02	1.6291357690e-03	2.5185158603e-02	6.0756242413e-02	2.2658154479e+00
6	1.1347305283e+00	6.5738367713e-05	6.3899421097e-06	1.6227458269e-03	3.9222894932e-03	2.4075890836e+00
7	1.1347241385e+00	1.0154053154e-09	9.8701713469e-11	6.3898434080e-06	1.5446417475e-05	2.4173016297e+00

ans = 1.134724138500221

The first column is simply an indicator of the current iteration of the algorithm. Since the last value in the column is 7, the algorithm converged in 7 steps with the given tolerance. With a lower tolerance, it would be expected to converge in more steps, the inverse being true as well. Column two consists of the root estimates at each iteration. Any row could be used as a solution to the problem, but the reliability and accuracy of the estimate depends on the other columns in the row. The estimates are accurate to more decimal places compared to the true solution at almost every step. The final estimate is accurate to 9 decimal places. The next column prints the functional value at the root estimate, and is decreasing at each step until it is very close to 0. This suggests that the problem $f(x) = 0$ is very close to being solved. Additionally, it should be noted that the functional value decreases rapidly at later steps, especially at the seventh step. This suggests the convergence of the algorithm accelerates at later iterations. It then follows that the fourth column for the absolute difference between α and current estimate x approaches 0, because the algorithm is producing estimates closer and closer to the true value. This column also suggests the acceleration of convergence as the values become closer to 0 at a faster rate for later iterations.

Column 5 shows the difference between the current estimate and the previous estimate. This is more applicable in real-world situations where the true solution is not known. The fact this column falls closer to 0 at each iteration means the estimate is not jumping around, rather it is converging to one value in particular. Like the previous two columns, this also decreases the most at the final step.

The ratios are slightly less intuitive to interpret, but both are representative of the convergence of Newton's Method. It is expected for $R_1 \rightarrow 0$ as $n \rightarrow \infty$ and $R_2 \rightarrow c$ as $n \rightarrow \infty$.

The former is easily explainable, as the algorithm relies upon each successive error being closer to zero than the previous error. That is, the estimate is closer to the true value at each iteration. Therefore, we expect the numerator $\alpha - x_n$ to be greater than the denominator $\alpha - x_{n-1}$, thus the limit as $n \rightarrow \infty$ of R_1 is 0. This result is seen in the table, with the value for the ratio in the last row being closest to 0. The convergence appears to accelerate based on this result as well.

The latter, R_2 , requires slightly more explanation. Taking the limit of the ratio as follows:

$$\lim_{n \rightarrow \infty} \frac{\alpha - x_n}{(\alpha - x_{n-1})^2} = \frac{-f''(\alpha)}{2f'(\alpha)},$$
 evaluated to be -2.417301 , which is nearly identical to the final value in the last column.

This is a result of the general definition of convergence order for iterative methods. Since the corresponding table approaches this specific value, the convergence of Newton's method can be confirmed here as well.

Also worth mentioning is the fact that repeating the code with lower values for "tol" gives the exact same output unless the tolerance is decreased to 0.000001, or 10^{-6} , at which point the algorithm continues for one more iteration. The resulting table is shown on the next page.

Interestingly, the behavior of each column follows the expected convergence order mentioned in the previous paragraphs *aside* from the second ratio, where it increases rapidly at the last step. This is almost certainly because the denominator of the term is extremely small, increasing the ratio overall. This only further supports the argument that the algorithm converges most strongly at the seventh iteration.

n	xn	f(xn)	True Err	Approx Err	R1	R2
0	2.0000000000e+00	6.1000000000e+01	8.6527586160e-01			
1	1.6806282723e+00	1.9852941124e+01	5.4590413385e-01	3.1937172775e-01	6.3090184076e-01	7.2913375810e-01
2	1.4307389882e+00	6.1467954843e+00	2.9601484984e-01	2.4988928401e-01	5.4224694682e-01	9.9330067900e-01
3	1.2549709561e+00	1.6516569690e+00	1.2024681771e-01	1.7576803213e-01	4.0621886967e-01	1.3722922005e+00
4	1.1615384328e+00	2.9430969112e-01	2.6814294372e-02	9.3432523336e-02	2.2299379628e-01	1.8544673409e+00
5	1.1363532742e+00	1.6826073996e-02	1.6291357690e-03	2.5185158603e-02	6.0756242413e-02	2.2658154479e+00
6	1.1347305283e+00	6.5738367713e-05	6.3899421097e-06	1.6227458269e-03	3.9222894932e-03	2.4075890836e+00
7	1.1347241385e+00	1.0154053154e-09	9.8701713469e-11	6.3898434080e-06	1.5446417475e-05	2.4173016297e+00
8	1.1347241384e+00	1.1102230246e-15	2.2204460493e-16	9.8701491424e-11	2.2496529910e-06	2.2792441103e+04

“=====”